

Преобразование текста в текст

Разработка эффективных подсказок

 Копировать страницу 

Разработка эффективных подсказок

Пишите четкие, конкретные инструкции

Расплывчатые подсказки дают расплывчатые результаты. Чем больше контекста и ограничений вы укажете, тем точнее результат будет соответствовать вашим ожиданиям. Представьте, что вы делегируете задачу: запрос в одну строку оставляет слишком много возможностей для интерпретации, в то время как подробные инструкции с четкими целями дают именно то, что вам нужно.

 Четкие и конкретные запросы — самый важный фактор для получения полезного результата от больших языковых моделей.

Пример: задача генерации кода

Неясный запрос

Напишите функцию на Python для обработки данных.

Четкий и конкретный запрос

Напишите функцию на Python, которая считывает CSV-файл, фильтрует строки, в которых столбец `status` равен `"active"`, и возвращает список словарей. Добавьте подсказки типов,

Пример: задача генерации контента

Неясный запрос

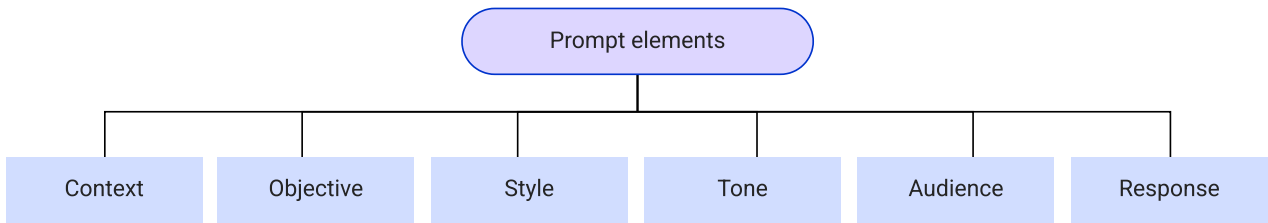
Четкий и конкретный запрос

Помогите мне написать анонс продукта.

Напишите анонс продукта «Zephyr Z9» — лёгкого портативного телефона от Bailian — объёмом 200 слов. Подчеркните его ультратонкий дизайн, высокую производительность и простоту использования. Ориентируйтесь на технически подкованных потребителей в социальных сетях. Пишите энергично и лаконично.

Структурируйте подсказки с помощью шаблона

Шаблон подсказки помогает систематически предоставлять информацию, необходимую для работы больших языковых моделей. Вместо того чтобы гадать, что включить в письмо, используйте эти шесть элементов в качестве чек-листа:



Элемент	Назначение	Example
Контекст	Справочная информация, имеющая отношение к задаче	"You are reviewing pull requests for a Python microservice."
Цель	Конкретная задача, которую необходимо выполнить	"Identify potential bugs and suggest fixes."
Стиль	Стиль письма или образ автора	"Write as a senior backend engineer."
Тон	Эмоциональная составляющая результата	"Professional but constructive."
Аудитория	Кто читает результат	"Junior developers on the team."
Ответ	Желаемый формат результата	"A numbered list of issues with code suggestions."

Используйте разделители # , чтобы обозначить каждый раздел в подсказке:

#Context#

You are reviewing pull requests for a Python microservice that handles user authentication. The codebase uses FastAPI and SQLAlchemy.

#Objective#

Review the following code diff and identify potential security vulnerabilities, performance issues, and style violations.

#Style#

Write as a senior backend engineer conducting a thorough code review.

#Tone#

Professional but constructive -- explain why each issue matters.

#Audience#

Junior developers who are learning secure coding practices.

#Response#

A numbered list of findings. For each finding, include:

- The specific line or code block
- Why it is a problem
- A suggested fix with code

 Не для каждой подсказки нужны все шесть элементов. Адаптируйте фреймворк под свою задачу. В примерах, приведенных в этом руководстве, используются разные подмножества в зависимости от сценария.

Optimize prompts

Provide output examples

Including examples of expected output helps the model match your format, style, and level of detail. This technique -- known as few-shot prompting -- also improves consistency across multiple runs.

Example: Structured data extraction

#Context#

Extract structured information from user support tickets.

#Objective#

Parse each ticket into a JSON object.

#Examples#

Input: "My order #12345 arrived damaged. I want a refund."

Output:

```
{
  "order_id": "12345",
  "issue_type": "damaged_item",
  "requested_action": "refund"
}
```

Input: "I can't log in to my account. I've tried resetting my password twice."

Output:

```
{
  "order_id": null,
  "issue_type": "login_failure",
  "requested_action": "account_recovery"
}
```

Модель изучает схему JSON, соглашения об именовании полей, а также способы определения `issue_type` и `requested_action` из естественного языка — и все это только на основе примеров.

Break complex tasks into steps

For complex tasks, define explicit steps for the model to follow -- not just the desired result. Step-by-step instructions prevent the model from skipping intermediate reasoning.

Example: Math word problem

#Context#

Timmy leaves home at 8:00 AM, walking at 50 meters per minute. After 12 minutes, his father notices Timmy forgot his homework and chases him on a bicycle at 200 meters per minute. After catching up, the father carries Timmy on the bicycle for the rest of the trip. The bicycle distance is 5 times the walking distance.

#Objective#

Calculate the arrival time.

#Steps#

1. Calculate how far Timmy walks before his father catches up.
2. Determine how long it takes the father to catch up.
3. Calculate the remaining bicycle distance and travel time.
4. Add up all time intervals to find the arrival time.

Результат работы модели (в соответствии с шагами):

Step 1: Timmy walks for 12 minutes before his father starts:

$12 \text{ min} * 50 \text{ m/min} = 600 \text{ m head start}$

Step 2: Father's relative speed: $200 - 50 = 150 \text{ m/min}$

Catch-up time: $600 \text{ m} / 150 \text{ m/min} = 4 \text{ minutes}$

Timmy walks an additional: $4 \text{ min} * 50 \text{ m/min} = 200 \text{ m}$

Total walking distance: $600 + 200 = 800 \text{ m}$

Step 3: Bicycle distance: $800 \text{ m} * 5 = 4,000 \text{ m}$

Bicycle time: $4,000 \text{ m} / 200 \text{ m/min} = 20 \text{ minutes}$

Step 4: Total time: $12 + 4 + 20 = 36 \text{ minutes}$

Arrival: $8:00 \text{ AM} + 36 \text{ min} = 8:36 \text{ AM}$

Без раздела #Steps# модель может пропустить промежуточные вычисления и выдать неверный ответ.

Use delimiters to separate content

When a prompt contains multiple blocks of content -- instructions, input data, and examples -- use delimiters to mark clear boundaries. This helps the model distinguish your instructions from the content it should process.

Use character sequences that rarely appear in natural language: ### , === , or >>> .

Example: Text summarization

Summarize each section of the following movie review separately.
Produce one summary sentence per section.

###

Zhang Zhiqiang was once spirited and energetic, but life's pressures pushed him off course. An impulsive decision in middle age transformed him from his family's pride into a social outcast.

###

This was just the beginning. A series of unexpected events forced him to confront himself. He chose to start over as a delivery driver, navigating city streets to rebuild his life.

###

Along the way, he found like-minded companions. Through their shared struggles, he rediscovered his courage and purpose, forging a new direction for his life.

###

Без разделителей модель воспринимает весь отзыв как единый блок и выдает одно общее резюме. С разделителями модель выделяет три отдельных раздела и генерирует три связанных между собой резюме, соответствующих повествовательной структуре.

Guide the model to reason step by step

For tasks involving logic, math, or multi-step analysis, have the model show its reasoning before stating a conclusion. Two effective techniques:

Chain of Thought (CoT)

CoT is a relatively simple method that can significantly improve the reasoning of LLMs in complex scenarios. Ask the model to explain its thinking before producing a final answer.

Example: JSON validation

```
#Context#
JSON input:
{"web-app": {
  "servlet": [
    {
      "servlet-name": "cofaxEmail",
      "servlet-class": "org.cofax.cds.EmailServlet",
      "init-param": {
        "mailHost": "mail1",
        "mailHostOverride": "mail2"
      },
    },
    {
      "servlet-name": "cofaxTools",
      "servlet-class": "org.cofax.cms.CofaxToolsServlet",
      "init-param": {
        "templatePath": "toolstemplates/",
        "log": 1,
        "logLocation": "/usr/local/tomcat/logs/CofaxTools.log",
        "logMaxSize": ""
      },
    },
    "servlet-mapping": {
      "cofaxEmail": "/cofaxutil/aemail/*",
      "cofaxTools": "/tools/*",
    },
    "taglib": {
```

Добавление пункта «объяснить свою логику» позволяет получить структурированный анализ, в котором модель оценивает каждое требование по отдельности, выявляя пограничные случаи (например, пустую строку в значении `logMaxSize`), которые она пропустила бы при выводе только в формате «да/нет».

Prompt chaining

Break a complex problem into a sequence of simpler questions across multiple turns. Each answer feeds into the next question, guiding the model along a controlled reasoning path.

Prompt chaining takes more effort to set up than CoT but performs better and has higher accuracy. It is particularly suitable for tasks that are logically complex but can be broken down according to fixed patterns.

Example: Multi-step math (same Timmy problem, three turns)

Turn 1:

#Context#

Timmy leaves at 8:00 AM walking at 50 m/min. After 12 minutes, his father chases on a bicycle at 200 m/min. The bicycle distance is 5 times the walking distance.

#Question#

When does the father catch Timmy, and how far has Timmy walked?

Модель: "Отец догоняет Тимми в 8:16 утра, после того как Тимми прошел 800 метров."

Поворот 2:

How far does Timmy travel by bicycle to reach his grandparents' house?

Модель: "Расстояние на велосипеде = $800 \text{ м} * 5 = 4000 \text{ метров}$."

Поворот 3:

What time does Timmy arrive?

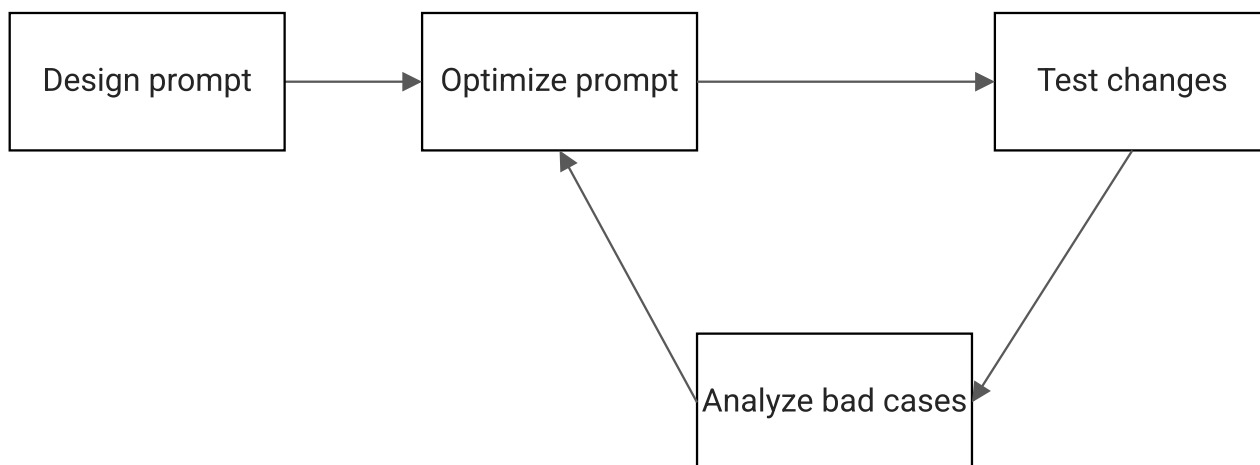
Модель: "Время на велосипеде = $4000 \text{ м} / 200 \text{ м/мин} = 20 \text{ минут}$. Общее время = $16 + 20 = 36 \text{ минут}$. Прибытие: 8:36. "

Каждый шаг основывается на проверенных промежуточных результатах, что снижает вероятность накопления ошибок.

- ① Other reasoning techniques include Tree of Thoughts (ToT) and Boosting of Thoughts. Choose the technique that matches your task complexity and accuracy requirements.

Test and iterate

Prompt engineering is iterative. Rarely does the first version of a prompt produce the best results.



Следуйте этому циклу:

1. **Define your goal.** Decide what good output looks like before writing the prompt.
2. **Write an initial prompt.** Apply the techniques from this guide: be specific, use a framework, and provide examples.
3. **Test with diverse inputs.** Run the prompt against multiple real-world scenarios, including edge cases.
4. **Analyze failures.** Identify where output falls short: wrong format, missing information, incorrect reasoning, or inconsistent tone.
5. **Refine the prompt.** Adjust one element at a time to measure the effect of each change.
6. **Repeat.** Continue until output consistently meets your quality bar.

Помимо разработки промпта, важна обратная связь от пользователей. После развертывания промпта в рабочей среде отслеживайте реальные взаимодействия и вносите изменения на основе закономерностей в исправлениях пользователей и непредвиденных ситуаций.

Example: Optimize a multilingual HR assistant

Background: A Qwen3.5-Flash-based HR assistant for a multinational company was not consistently responding in the user's language. The system prompt needed restructuring.

Before optimization

Role

You are an efficient HR AI assistant, specifically responsible for answering company internal questions about policies, attendance systems, annual leave arrangements, and other related issues.

Skills

Skill 1: Policy interpretation

- Accurately interpret company policy documents and provide clear, concise policy explanations to colleagues.

Skill 2: Attendance Q&A

- Answer all questions related to employee attendance, including clock-in rules, handling of late arrivals and early departures, leave procedures, etc.

Skill 3: Annual leave management consultation

- Explain annual leave application conditions, accumulation rules, validity period, and approval process.

Limitations

- Respond in the same language as the user's question.
- Limited to HR-related questions.

Проблемы:

- Loose structure with a redundant `## Limitations` section
- The `${documents}` knowledge base block was embedded inline, making it hard for the model to distinguish instructions from reference content
- Language-matching instructions were buried in limitations

After optimization

#Context#

You are an HR AI assistant for a multinational company. You answer questions about policy interpretation, attendance rules, and annual leave management.

Below are the company policy documents:

=====

`${documents}`

=====

#Objective#

1. Only answer questions about policy interpretation, attendance, and annual leave.
2. When a question is in scope but the knowledge base does not cover it, direct the user to contact the HR department.
3. For each category, follow these guidelines:
 - Policy interpretation: Locate the relevant clause and provide a clear, concise explanation.
 - Attendance: Cover clock-in rules, late arrival handling, and leave procedures.
 - Annual leave: Explain eligibility, accrual rules, approval process, and help calculate remaining balance.

Что изменилось:

- **Framework applied.** Restructured using `#Context#` , `#Objective#` , `#Multilingual requirements#` , and `#Response#` sections.
- **Delimiters added.** Wrapped `${documents}` in `=====` delimiters to separate reference data from instructions.
- **Constraints distributed.** Moved constraints into their relevant sections (language rules into `#Multilingual requirements#` , scope limits into `#Objective#`).

Эти изменения решили проблему согласованности языка, поскольку модель могла четко идентифицировать многоязычную инструкцию как отдельную, приоритетную директиву, а не как второстепенный элемент.





Products

[Cloud](#)

[Private](#)

[Policy](#)

[Terms
of
Use](#)

[Customer
Agreement](#)

[Trust
Center](#)

[Changelog](#)

© 2026 Intelligent
Cloud Computing
(Singapore) Private
Limited. All rights
reserved.
[Cookie preferences](#)